

Structranet AI GNS3 Static Project Export Architecture

Defining the structural requirements for pivoting from a live REST API deployment model to a Static Portable Project (.gns3project) Export Model, including the complete GNS3 project serialization blueprint, hardware injection logic, and offline-first data flow pipeline.

CLASSIFICATION
Architectural Blueprint

TARGET PLATFORM
GNS3 2.2.x

REVISION
1.0

Table of Contents

1. Executive Summary	3
2. The .gns3project Archive Anatomy	3
2.1 Top-Level Archive Structure	3
2.2 Export Exclusion Rules	4
2.3 Portable vs. Regular Project Differences	4
3. Minimum Valid "Portable Project" State	5
3.1 Top-Level Required Fields	5
3.2 Minimal Valid project.gns3 Example	6
3.3 Revision History and Compatibility	7
4. Node Property Requirements by Appliance Type	7
4.1 Valid Node Types and Categories	7
4.2 Dynamips Node Properties	8
4.2.1 Platform-to-Adapter Mapping	9
4.3 IOU Node Properties	9
4.4 VPCS Node Properties	10
4.5 QEMU Node Properties	10
5. Hardware Injection Logic	11
5.1 Port Addressing by Node Type	12
5.2 Platform-Specific Interface Naming	12
5.3 Idempotent Link Wiring Algorithm	13
6. UUID Consistency and File-System Mapping	14
6.1 Node ID to Directory Mapping	14
6.2 Config Content vs. Config Path	15

6.3 UUID Regeneration During Import	15
7. Coordinate and Visual Logic	16
7.1 Canvas Coordinate System	16
7.2 Symbol Path Conventions	17
7.3 Label Positioning	17
8. Redesigned Data Flow: Topology Skeleton to GNS3-Compliant Digital Twin.	
8.1 Stage 1: Topology Skeleton Ingestion	18
8.2 Stage 2: Node Enrichment and UUID Assignment	18
8.3 Stage 3: Link Resolution and Port Allocation	18
8.4 Stage 4: Canvas Layout Computation	19
8.5 Stage 5: Configuration Generation and Injection	19
8.6 Stage 6: project.gns3 JSON Assembly	19
8.7 Stage 7: Archive Packaging and Verification	19
8.8 Data Flow Summary	20
9. Import Validation Checkpoints	20
10. Architectural Constraints and Recommendations	21
10.1 Offline-First Constraint	21
10.2 Determinism Constraint	22
10.3 Schema Evolution Considerations	22
10.4 Error Recovery Strategy	22
10.5 Testing Strategy	22
Appendix A: Complete Node Object Reference	23
Appendix B: Complete Link Object Reference	24
Appendix C: Dynamips Valid Adapter Modules	25

1. Executive Summary

This Technical Specification Document defines the architectural blueprint for pivoting the Structranet AI platform from a live REST API deployment model to a Static Portable Project (.gns3project) Export Model. The core objective is to enable the offline generation of valid, importable GNS3 project archives without requiring connectivity to a running GNS3 server instance at any point during the creation or export process. This represents a fundamental shift in the platform paradigm: rather than orchestrating live compute resources, the system must now construct a self-contained digital twin of a network topology that can be transported across environments and instantiated on any GNS3 installation.

The specification addresses four critical domains that must be solved for this pivot to succeed. First, the **anatomy of the .gns3project archive** must be understood at a forensic level, including the exact relationship between the project.gns3 JSON manifest and the project-files directory hierarchy. Second, the **minimum valid state** of a portable project JSON must be defined with precision, enumerating every mandatory field, default value, and inter-field dependency that GNS3 validates during import. Third, the **hardware injection logic** must map logical topology links to physical adapter slots and port numbers in a way that respects platform-specific naming conventions (such as FastEthernet0/0 versus Ethernet0/0) and ensures idempotent behavior across generation runs. Fourth, the **end-to-end data flow** from an AI-generated topology skeleton to a GNS3-compliant .gns3project ZIP must be designed as a deterministic, offline-first pipeline that guarantees 100% import success on any compliant GNS3 2.2+ server.

All architectural guidance in this document is derived from analysis of the GNS3 server source code (gns3-server v2.2.x), the official GNS3 file format schema (gns3_file.json), and the internal export/import pipeline modules (export_project.py, import_project.py). No source code modifications are proposed or required; this specification provides the structural knowledge necessary to construct valid project artifacts purely from static data.

2. The .gns3project Archive Anatomy

A .gns3project file (also recognized with the .gns3p extension) is a standard ZIP archive that encapsulates the complete state of a GNS3 network topology. The archive serves as a portable, self-contained unit that can be imported into any GNS3 installation regardless of the host operating system or compute configuration. Understanding the exact structure of this archive is foundational to constructing valid projects offline, as any deviation from the expected layout will result in import failures or silently corrupted topologies.

2.1 Top-Level Archive Structure

The archive must contain, at minimum, a file named **project.gns3** at its root. This is the topology manifest that GNS3 reads first during import. Alongside it, the **project-files/** directory tree stores all per-node working files such as startup configurations, NVRAM binaries, and disk images. An optional

images/ directory may contain binary OS images (IOS, IOU, QEMU disk images) when the project is exported with `include_images=True`. The following listing shows the canonical directory structure:

```
my_project.gns3project/
+-- project.gns3 # REQUIRED: topology JSON manifest
+-- project-files/ # Per-node working files
| +-- dynamips// # Dynamips node data
| | +-- configs/
| | | +-- startup-config.cfg # IOS startup configuration
| | | +-- private-config.cfg # IOS private configuration
| | +-- nvram # NVRAM binary
| | +-- disk0 # Disk0 binary
| | +-- disk1 # Disk1 binary
| +-- iou// # IOU node data
| | +-- startup-config.cfg # IOU startup config
| | +-- private-config.cfg # IOU private config
| | +-- nvram_0000b000 # IOU NVRAM file
| +-- vpcs// # VPCS node data
| | +-- startup.vpc # VPCS startup script
| +-- qemu// # QEMU node data
| | +-- hda_disk.qcow2 # QEMU disk images
| +-- docker// # Docker node data
| +-- captures/ # Excluded from export
| +-- snapshots/ # Excluded by default
| +-- tmp/ # Excluded from export
+-- images/ # OPTIONAL: binary OS images
+-- IOS/ # Dynamips IOS images
+-- IOU/ # IOU images
+-- QEMU/ # QEMU disk images
```

2.2 Export Exclusion Rules

The GNS3 export pipeline (`export_project.py`) enforces strict exclusion rules that determine which files are included in or omitted from the `.gns3project` archive. These rules must be mirrored by any offline generation system to ensure that the produced archives are consistent with what GNS3 itself would generate. The `project.gns3` file is always processed first and written to the archive root. Files under `project-files/tmp/` and `project-files/captures/` are always excluded. The `project-files/snapshots/` directory is excluded unless `include_snapshots` is explicitly `True`. Files ending in `_log.txt`, `.log`, or `.DS_Store` are excluded regardless of their location. Binary images are optionally included under `images//` when the `include_images` flag is set. Empty directories within `project-files/` are preserved in the archive. Additionally, any file modification time before 1980 is patched upward, as the ZIP format cannot represent timestamps earlier than that year.

2.3 Portable vs. Regular Project Differences

A portable project differs from a regular GNS3 project in several critical ways that directly impact the offline generation architecture. The most important distinction is that all `compute_id` values must be set to "local" rather than referencing remote compute servers such as "vm" or custom endpoints. The

topology.computes array must be stripped to an empty array. Image paths must use only the basename of the file (e.g., "c3725-adventerprisek9-mz.124-15.T14.image") rather than absolute filesystem paths. Packet capture files and log files are excluded entirely. These transformations are applied by the `_patch_project_file` function during export and must be replicated during offline generation to produce a project that GNS3 recognizes as portable.

Aspect	Regular Project	Portable Project (.gns3project)
compute_id	May be "vm", "local", or custom	All set to "local"
topology.computes	Contains all compute entries	Stripped to empty array []
Image paths	Full absolute paths	Basename only
Captures	Present in project-files	Excluded
Snapshots	May be present	Excluded by default
Log files	May be present	Excluded

3. Minimum Valid "Portable Project" State

This section defines the absolute minimum JSON state that a `project.gns3` file must contain to be considered a valid portable project by the GNS3 import pipeline. The validation logic in `import_project.py` performs several checks during import: it verifies ZIP integrity, confirms the existence of `project.gns3`, validates JSON parseability, and requires the presence of `project_id` and `name` keys at the top level. Any failure at these checkpoints results in an immediate import error. The following specification enumerates every required field, its type constraints, and its default value when applicable.

3.1 Top-Level Required Fields

The `project.gns3` JSON file must contain the following top-level keys to pass import validation. The `project_id` field must be a valid UUID in the canonical 8-4-4-4-12 hexadecimal format. The `type` field must always be the string "topology". The `revision` field indicates the file format version and must be set to 9 for GNS3 2.2.x compatibility. The `version` field should reflect the GNS3 software version, typically "2.2.52" or equivalent. The `name` field must not contain unsupported characters that would cause a `UnicodeEncodeError`. The `topology` object must be present and must contain four required sub-arrays: `computes`, `drawings`, `links`, and `nodes` (all of which may be empty).

Key	Type	Required	Default / Constraint
project_id	string (UUID)	Yes	Pattern: <code>^[a-fA-F0-9]{8}-...-[a-fA-F0-9]{12}\$</code>
type	enum	Yes	Must be "topology"

revision	integer	Yes	9 (for GNS3 2.2.x)
version	string	Yes	e.g., "2.2.52"
name	string	Yes	No unsupported characters
topology	object	Yes	Must contain computes, drawings, links, nodes
auto_close	boolean	No	true
auto_open	boolean	No	false
auto_start	boolean	No	false
scene_height	integer	No	1000
scene_width	integer	No	2000
zoom	integer	No	100
show_interface_labels	boolean	No	false
grid_size	integer	No	75
drawing_grid_size	integer	No	25
supplier	object/null	No	null
variables	array/null	No	null

3.2 Minimal Valid project.gns3 Example

The following JSON represents the absolute minimum valid state for a portable project.gns3 file. While additional fields may be included, omitting any of the fields shown below will result in an import failure. The topology.computes array must be empty for portable projects, and the topology.nodes and topology.links arrays will be populated based on the generated topology skeleton.

```
{
  "auto_close": true,
  "auto_open": false,
  "auto_start": false,
  "drawing_grid_size": 25,
  "grid_size": 75,
  "name": "My Project",
  "project_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "revision": 9,
  "scene_height": 1000,
  "scene_width": 2000,
  "show_interface_labels": false,
  "topology": {
    "computes": [],
    "drawings": [],
```

```
"links": [],
"nodes": []
},
"type": "topology",
"version": "2.2.52",
"zoom": 100
}
```

3.3 Revision History and Compatibility

The revision field in the project.gns3 file indicates the file format version and directly affects how GNS3 processes the project during import. For the Structranet AI platform, targeting revision 9 (GNS3 2.2.x) is the correct choice, as this is the current stable format. Earlier revisions (5 through 8) may lack support for features required by modern node types. It is critical not to mix revision values with version string values; the revision field is an integer, while the version field is a semver string representing the GNS3 software version that last updated the file.

Revision	GNS3 Version	Notes
9	2.2.x	Current stable format (target for Structranet AI)
8	2.1.x	Legacy format
7	2.0	Early 2.0 format
5	2.0 alpha	Pre-release format
4	1.5	Legacy 1.x format

4. Node Property Requirements by Appliance Type

Each node in the topology.nodes array must conform to a strict schema determined by its node_type. The GNS3 server validates node properties during both creation and import, and missing mandatory fields will cause the node to be rejected. This section provides the complete property specification for each appliance type that the Structranet AI platform is expected to support, including Dynamips (Cisco IOS routers), IOU (Cisco IOS on UNIX), VPCS (Virtual PC Simulator), and QEMU (virtual machines). Each specification includes the mandatory creation fields, the complete property set with defaults, and the valid values for constrained fields.

4.1 Valid Node Types and Categories

The node_type field in each node object determines which schema validator is applied. The following table lists all valid node_type values along with their associated categories. The category field can be either a string enum ("router", "switch", "guest", "firewall") or a legacy integer value (0=router,

1=switch, 2=guest, 3=firewall). For the Structranet AI platform, the string enum form should always be used. The symbol field maps to built-in SVG resources within GNS3 and must be selected based on both the node_type and category to ensure correct visual rendering.

node_type	Category	Default Symbol	Description
dynamips	router	:/symbols/router.svg	Cisco IOS via Dynamips emulation
iou	router / switch	:/symbols/multilayer_switch.svg	Cisco IOS on UNIX (L3/L2)
vpcs	guest	:/symbols/vpcs_guest.svg	Virtual PC Simulator (lightweight PC)
qemu	guest	:/symbols/qemu_guest.svg	QEMU virtual machine
docker	guest	:/symbols/docker_guest.svg	Docker container node
ethernet_switch	switch	:/symbols/ethernet_switch.svg	Built-in Ethernet switch
ethernet_hub	switch	:/symbols/hub.svg	Built-in Ethernet hub
cloud	guest	:/symbols/cloud.svg	Host interface cloud node
nat	guest	:/symbols/nat.svg	NAT node

4.2 Dynamips Node Properties

Dynamips nodes represent Cisco IOS routers and are the most complex node type in terms of required properties. The mandatory creation fields are name, platform, image, and ram. The platform field must match the pattern `^c[0-9]{4}$` and determines which adapter modules are available for the slot0 through slot6 fields. The image field specifies the IOS image filename (basename only for portable projects). The ram field specifies the allocated RAM in megabytes and has platform-specific defaults: 128MB for c1700, 192MB for c2600, 256MB for c3725/c3745/c2691, and 512MB for c7200.

The slot configuration is the primary mechanism for defining which network interfaces a Dynamips router possesses. Each slot (slot0 through slot6, depending on platform) can accept one adapter module. The available modules vary by platform: the c7200 uses Port Adapters (PA-*) in slots 1-6 with a fixed C7200-IO-FE or C7200-IO-2FE in slot 0, while the c3725/c3745 use Network Modules (NM-*) in slots 0-1 with a fixed GT96100-FE or Leopard-2FE in the motherboard slot. The WIC slots (wic0-wic2) provide additional serial or ethernet interfaces on platforms that support them. Understanding this slot-to-interface mapping is essential for correctly wiring links in the topology.

Property	Type	Required	Default	Description
platform	string	Yes	-	c7200, c3745, c3725, c3600, c2691, c2600, c1700
image	string	Yes	-	IOS image filename (basename for portable)

ram	integer	Yes	varies	RAM in MB (128-512 depending on platform)
nvrn	integer	No	128-512	NVRAM in KB
startup_con fig	string	No	ios_base_ startup-co nfig.txt	Startup config filename
private_con fig	string	No	""	Private config filename
slot0-slot6	string/null	No	null	Network module or port adapter
wic0-wic2	string/null	No	null	WIC module (WIC-1ENET, WIC-1T, WIC-2T)
idlepc	string	No	""	Idle PC value for CPU optimization
mac_addr	string/null	No	""	MAC address pattern
mmap	boolean	No	true	Use memory-mapped files
sparsemem	boolean	No	true	Use sparse memory
npe	enum	No	npe-400	C7200 only: NPE type
chassis	string	No	-	Pattern: ^[0-9]{4}(XM)?\$

4.2.1 Platform-to-Adapter Mapping

The available adapter modules vary significantly by platform, and incorrect slot assignments will cause GNS3 to reject the node or produce non-functional interfaces. The c7200 platform uses Port Adapters (PA-*) in slots 1 through 6, with a fixed I/O controller in slot 0 (C7200-IO-FE, C7200-IO-2FE, or C7200-IO-GE-E). The c3725 and c3745 platforms use Network Modules (NM-*) in slots 0 and 1, with a built-in GT96100-FE (c3725) or Leopard-2FE (c3745) on the motherboard that provides two FastEthernet interfaces as adapter 0. The c2600 platform supports NM-* and C2600-MB-* modules. The c1700 platform supports a limited set including C1700-MB-* and NM-1FE-TX. When constructing a Dynamips node statically, the Structranet AI system must validate that the selected module is compatible with the chosen platform before emitting the node JSON.

4.3 IOU Node Properties

IOU (IOS on UNIX) nodes are simpler than Dynamips nodes in terms of hardware configuration but have their own unique requirements. The mandatory creation fields are name, path, and application_id. The path field specifies the IOU binary filename (basename only for portable projects). The application_id is an integer that must be unique across all IOU nodes in the project; it is used by the IOU process to identify itself. The ethernet_adapters and serial_adapters fields define the number of interface bays, each providing 4 ports. This means an IOU node with 2 ethernet adapters has 8 ethernet ports (Ethernet0/0 through Ethernet0/3 and Ethernet1/0 through Ethernet1/3), and similarly for serial adapters.

Property	Type	Required	Default	Description
path	string	Yes	-	IOU binary filename (basename for portable)
application_id	integer	Yes	auto	Unique application ID for IOU process
ethernet_adapters	integer	No	2	Number of ethernet adapter bays (each = 4 ports)
serial_adapters	integer	No	2	Number of serial adapter bays (each = 4 ports)
ram	integer/null	No	1024	RAM in MB
nvrाम	integer/null	No	256	NVRAM in KB
l1_keeplives	boolean/null	No	false	Always-up ethernet interface
startup_config	string	No	iou_l3_base_startup-config.txt	Config filename
use_default_iou_values	boolean/null	No	false	Use default IOU values

4.4 VPCS Node Properties

VPCS (Virtual PC Simulator) nodes are the simplest node type, requiring only a name for creation. Each VPCS node has exactly one ethernet adapter with one interface, which means its adapter_number is always 0 and port_number is always 0. The startup_script property allows inline specification of VPCS commands that are written to the project-files/vpcs//startup.vpc file upon node creation. This is the primary mechanism for injecting IP address configurations and other VPCS commands into the node without requiring the user to manually configure them after import.

Property	Type	Required	Default	Description
name	string	Yes	-	VPCS node name
console	integer/null	No	auto	Console TCP port
console_type	enum	No	"telnet"	telnet or none
startup_script	string/null	No	-	VPCS startup script content

4.5 QEMU Node Properties

QEMU nodes are the most flexible appliance type, supporting 29 different CPU platforms and a wide range of adapter types. The mandatory creation field is only name, but for a functional node, the `qemu_path`, `platform`, `ram`, and at least one disk image should be specified. The `adapters` field controls the number of network interface cards, each providing exactly one port (`adapter_number` = NIC index, `port_number` = always 0). The `adapter_type` field determines the emulated NIC hardware, with `e1000` being the default and most widely compatible choice. The `linked_clone` field should be set to true for portable projects to ensure that disk images are managed as linked clones rather than full copies.

Property	Type	Required	Default	Description
name	string	Yes	-	VM name
qemu_path	string/null	No	""	Path to QEMU binary
platform	enum	No	""	29 platforms (x86_64, i386, arm, aarch64, etc.)
linked_clone	boolean	No	true	Use linked clone for disk management
ram	integer/null	No	256	RAM in MB
cpus	integer/null	No	1	Number of vCPUs (1-255)
adapters	integer/null	No	1	Number of NICs (0-275)
adapter_type	string	No	"e1000"	NIC emulation type (e1000, virtio, etc.)
hda_disk_image	string	No	""	Primary disk image path
hda_disk_interface	enum	No	"none"	ide, sata, nvme, scsi, virtio, none
console_type	enum	No	"telnet"	telnet, vnc, spice, spice+agent, none
custom_adapters	array	No	[]	Per-adapter custom configuration

5. Hardware Injection Logic

The hardware injection logic defines how logical topology links are mapped to physical hardware slots, modules, and ports in a static environment. This is one of the most critical aspects of offline project generation, as incorrect port mapping will result in links that fail to establish or connect to the wrong interfaces. The challenge is that different node types use different addressing schemes: Dynamips uses slot numbers and port indices, IOU uses bay numbers and per-bay port indices, VPCS uses a single fixed interface, and QEMU uses adapter indices with a single port per adapter. The following specification defines the most efficient, idempotent logic for mapping logical links to physical hardware in each case.

5.1 Port Addressing by Node Type

Every link in a GNS3 topology connects exactly two endpoints, where each endpoint specifies a `node_id`, `adapter_number`, and `port_number`. The interpretation of these fields depends entirely on the `node_type` of the referenced node. For Dynamips nodes, the `adapter_number` corresponds to the slot index (0-6), and the `port_number` corresponds to the port index within the module installed in that slot. For IOU nodes, the `adapter_number` corresponds to the bay index, and each bay provides exactly 4 ports (`port_number` 0-3). For VPCS nodes, both `adapter_number` and `port_number` are always 0. For QEMU nodes, the `adapter_number` corresponds to the NIC index, and `port_number` is always 0 because each QEMU adapter provides exactly one interface. The following table summarizes these differences:

Node Type	adapter_number	port_number	Maps To
Dynamips	Slot index (0-6)	Port on module	e.g., slot0=NM-1FE-TX: adapter 0, port 0 = FastEthernet0/0
IOU	Bay index	Port in bay (0-3)	adapter 0 port 0 = Ethernet0/0; adapter 1 port 0 = Ethernet1/0
VPCS	Always 0	Always 0	Single ethernet interface (eth0)
QEMU	NIC index (0-N)	Always 0	One port per adapter; adapter 0 = first NIC
Ethernet Switch	Always 0	Port number (0-N)	Switch port index
Ethernet Hub	Always 0	Port number (0-N)	Hub port index

5.2 Platform-Specific Interface Naming

One of the most common pitfalls in static topology generation is producing interface names that do not match the naming convention expected by the platform. Cisco IOS on Dynamips uses interface names like FastEthernet0/0, FastEthernet0/1, Serial0/0, and Serial0/1, where the first number is the slot/module index and the second number is the port index within that module. The exact name depends on the module type: NM-1FE-TX produces "FastEthernet" interfaces, NM-4E produces "Ethernet" interfaces, and NM-4T produces "Serial" interfaces. On the c7200 platform, PA-GE produces "GigabitEthernet" interfaces, while PA-FE-TX produces "FastEthernet" interfaces.

IOU nodes use a different naming convention: Ethernet0/0 through Ethernet0/3 for the first bay, Ethernet1/0 through Ethernet1/3 for the second bay, and so on. Serial interfaces follow the same pattern: Serial0/0 through Serial0/3. The `port_name_format` field on the node object controls how GNS3 renders interface labels; for IOU it defaults to "Ethernet{segment0}/{port0}" with `port_segment_size=4`, which produces the correct naming. For Dynamips, the default `port_name_format` is "Ethernet{0}" which produces names like "Ethernet0", "Ethernet1", etc. The Structranet AI system must generate the correct

port_name_format and port_segment_size values for each node type to ensure that link labels are rendered correctly on the GNS3 canvas.

Platform	Module	Interface Prefix	Example Name	Format Pattern
Dynamips (c3725)	GT96100-FE (built-in)	FastEthernet	FastEthernet0/0	FastEthernet{0}/{1}
Dynamips (c3725)	NM-1FE-TX	FastEthernet	FastEthernet1/0	FastEthernet{0}/{1}
Dynamips (c3725)	NM-4E	Ethernet	Ethernet1/0	Ethernet{0}/{1}
Dynamips (c3725)	NM-4T	Serial	Serial1/0	Serial{0}/{1}
Dynamips (c7200)	PA-GE	GigabitEthernet	GigabitEthernet1/0	GigabitEthernet{0}/{1}
IOU (L3)	Bay 0	Ethernet	Ethernet0/0	Ethernet{segment0}/{port0}
IOU (L3)	Bay 0	Serial	Serial0/0	Serial{segment0}/{port0}
VPCS	Single NIC	Ethernet	Ethernet0	Ethernet{0}
QEMU	adapter_type=e1000	Ethernet	Ethernet0	Ethernet{0}

5.3 Idempotent Link Wiring Algorithm

To ensure that the same topology skeleton always produces the same .gns3project output, the link wiring algorithm must be deterministic and idempotent. This means that given the same input topology definition, the system must always produce identical adapter_number and port_number assignments for each link endpoint, regardless of the order in which nodes are processed or links are created. The recommended approach is to maintain a per-node port allocation counter that tracks the next available adapter and port index for each node type. When a link is created, each endpoint is allocated the next available port on its respective node, and the counter is advanced. This ensures that port assignments are deterministic and that no two links are assigned to the same physical interface.

The allocation logic must respect the physical constraints of each node type. For Dynamips nodes, the algorithm must check which slots have modules installed and allocate ports sequentially within each occupied slot before moving to the next slot. For IOU nodes, ports are allocated sequentially within each bay (0-3), and a new bay is used only when the current bay is full. For QEMU nodes, each new link increments the adapter_number, and port_number is always 0. For VPCS nodes, only one link is possible, and any attempt to create a second link must be flagged as an error during topology validation. The following pseudocode illustrates the core allocation logic:

```

# Per-node port allocation state
port_counters = {
    node_id: {
        "current_adapter": 0,
        "current_port": 0,
        "max_ports_per_adapter":
    }
}

def allocate_port(node_id, node_type):
    state = port_counters[node_id]
    adapter = state["current_adapter"]
    port = state["current_port"]
    max = state["max_ports_per_adapter"]

    state["current_port"] += 1
    if state["current_port"] >= max:
        state["current_port"] = 0
        state["current_adapter"] += 1

    return adapter, port

# max_ports_per_adapter values:
# Dynamips: varies by module (1-8 ports per slot)
# IOU: 4 ports per bay
# VPCS: 1 total (no additional allocation possible)
# QEMU: 1 port per adapter

```

6. UUID Consistency and File-System Mapping

The integrity of a GNS3 project depends on strict consistency between the UUIDs referenced in the project.gns3 JSON and the directory names used in the project-files hierarchy. Every node has a `node_id` (UUID) that serves as the directory name under `project-files//`. Every link has a `link_id` (UUID) that uniquely identifies the connection. During import, GNS3 regenerates all UUIDs and renames the corresponding directories to match the new IDs. Understanding this regeneration behavior is critical for offline generation, because the system must ensure that the UUID-to-directory mapping is consistent at the point of archive creation, even though the UUIDs will be replaced upon import.

6.1 Node ID to Directory Mapping

The mapping between node IDs and filesystem directories is straightforward but must be followed precisely. Each node's `node_id` (a standard UUID string) becomes the directory name under the appropriate node-type subdirectory. For example, a Dynamips node with `node_id` "a1e920ca-338a-4e9f-b363-aa607b09dd80" will have its startup configuration stored at `project-files/dynamips/a1e920ca-338a-4e9f-b363-aa607b09dd80/configs/startup-config.cfg`. An IOU node with the same ID would store its configuration at

project-files/iou/a1e920ca-338a-4e9f-b363-aa607b09dd80/startup-config.cfg. A VPCS node would store its startup script at project-files/vpcs/a1e920ca-338a-4e9f-b363-aa607b09dd80/startup.vpc. The key insight is that the node_type in the JSON determines the subdirectory name, not the appliance category or any other field.

Node Type	Config Directory	Config Filename	Additional Files
dynamips	project-files/dynamips//configs /	startup-config.cfg	private-config.cfg, nvram, disk0, disk1
iou	project-files/iou//	startup-config.cfg	private-config.cfg, nvram_0000b000
vpcs	project-files/vpcs//	startup.vpc	None
qemu	project-files/qemu//	N/A (disk-based)	hda_disk.qcow2, hdb_disk.qcow2, config.img
docker	project-files/docker//	N/A	Container files

6.2 Config Content vs. Config Path

GNS3 supports two distinct mechanisms for injecting configurations into nodes, and the Structranet AI platform must understand the difference to choose the correct approach for offline generation. The first mechanism is **inline config content**, where the node properties contain startup_config_content or private_config_content fields with the full configuration text. When the node is started, GNS3 writes this content to the appropriate file in the project-files directory. The second mechanism is **config file reference**, where the startup_config or private_config fields reference files in the GNS3 global configs directory (~/.config/GNS3/configs/), not in the project-files directory.

For the Structranet AI offline generation model, the recommended approach is to use the file-system injection method: write the configuration content directly into the appropriate files within the project-files directory structure of the ZIP archive. This avoids the dependency on global config files that may not exist on the target GNS3 installation. The startup_config_content and private_config_content fields in the node properties should be populated with the AI-generated configuration text, and the corresponding files should be placed in the correct locations within the archive. When GNS3 imports the project and starts a node, it will detect the existing configuration files in project-files and use them, regardless of the startup_config property value.

6.3 UUID Regeneration During Import

When a .gns3project archive is imported, the GNS3 server regenerates all UUIDs through the regenerate_topology_ids() function in import_project.py. This process assigns new node IDs to every node, new link IDs to every link, and new drawing IDs to every drawing. A mapping table (old_id to new_id) is maintained and applied consistently across all references. The directory files in project-files/ are renamed from / to / using the _move_node_file() function. Link endpoint node_id references are

updated to the new IDs. This means that the specific UUID values chosen during offline generation are transient; they need only be internally consistent within the archive. However, they must be valid UUIDs (matching the canonical format) and must be unique within their respective domains (no two nodes may share the same `node_id`, no two links may share the same `link_id`).

Note: The only exception to UUID regeneration is when restoring a project snapshot, where IDs must match the parent project. This scenario is not applicable to the Structranet AI offline generation model.

7. Coordinate and Visual Logic

The visual rendering of a GNS3 topology on the canvas is governed by the `x`, `y`, `z`, `symbol`, and `label` fields of each node object. Incorrect values will result in overlapping nodes, missing icons, or unreadable labels. This section specifies the coordinate system, symbol path conventions, and label positioning rules that the Structranet AI system must follow to produce topologies that render correctly on import.

7.1 Canvas Coordinate System

The GNS3 canvas uses a standard Cartesian coordinate system with the origin (0, 0) at the center of the drawing area. Positive `X` extends to the right, negative `X` extends to the left, positive `Y` extends downward, and negative `Y` extends upward. The default scene dimensions are `scene_width=2000` and `scene_height=1000`, which means the visible canvas area extends from approximately (-1000, -500) to (1000, 500) in coordinate space. Nodes are typically spaced 100-200 pixels apart to avoid visual overlap. The `z` field controls the stacking order, with higher values rendering on top of lower values; the default is `z=1` for all nodes.

For the Structranet AI system, a deterministic layout algorithm is recommended that places nodes based on their logical topology role. Routers should be placed in a central horizontal band, switches below and above them, and end devices (VPCS, QEMU) at the periphery. A simple grid-based layout with 200px horizontal and 150px vertical spacing produces clean, readable topologies. The following table provides recommended default positions for common topology patterns:

Topology Pattern	Router Position	Switch Position	End Device Position
Hub-and-Spoke	Center (0, 0)	Periphery (radius 300)	Outer ring (radius 500)
Linear Chain	Horizontal (-400, 0), (-200, 0), (0, 0)...	Between routers	End of chain
Three-Tier	Core (0, -200)	Distribution (0, 0)	Access (0, 200)
Mesh (4-node)	Corners of 400x400 square	N/A	Outside each router

7.2 Symbol Path Conventions

The symbol field on each node determines which SVG icon is rendered on the canvas. GNS3 uses a resource path syntax where built-in symbols are referenced with the `"/symbols/"` prefix. The canonical symbol paths for the Classic theme are listed in the following table. If a symbol cannot be found, GNS3 falls back to `"/symbols/classic/computer.svg"`. The Structranet AI system must use the correct symbol path for each node type and category combination to ensure professional-looking topology diagrams. Custom symbols can be provided as relative or absolute SVG paths, but for maximum portability, built-in symbols should be preferred.

Symbol Key	Classic Path	Default For
router	<code>:/symbols/router.svg</code>	Dynamips routers, some IOU L3
multilayer_switch	<code>:/symbols/multilayer_switch.svg</code>	IOU L3 switches
ethernet_switch	<code>:/symbols/ethernet_switch.svg</code>	Ethernet switch nodes
vpcs_guest	<code>:/symbols/vpcs_guest.svg</code>	VPCS nodes
qemu_guest	<code>:/symbols/qemu_guest.svg</code>	QEMU VMs
firewall	<code>:/symbols/firewall.svg</code>	Firewall appliances (ASA, etc.)
cloud	<code>:/symbols/cloud.svg</code>	Cloud nodes
nat	<code>:/symbols/nat.svg</code>	NAT nodes
computer	<code>:/symbols/computer.svg</code>	Generic fallback

7.3 Label Positioning

Each node has a label object that controls the display of the node name on the canvas. The label consists of four fields: text (the display string), style (an SVG CSS style attribute), x and y (offsets from the node center), and rotation (degrees, -359 to 360). The default style is `"font-family: TypeWriter;font-size: 10.0;font-weight: bold;fill: #000000;fill-opacity: 1.0;"`. The x and y offsets are relative to the node's center position: negative x moves the label left, negative y moves it up. For standard node sizes (width=56, height=40 for routers), a label offset of x=-17, y=-25 places the label above and slightly left of the node center, which is the conventional position. For VPCS nodes (width=34, height=32), the offset is typically x=-8, y=-22.

8. Redesigned Data Flow: Topology Skeleton to GNS3-Compliant Digital Twin

This section proposes the high-level data flow that transforms an AI-generated topology skeleton into a fully GNS3-compliant `.gns3project` archive. The pipeline is designed as a deterministic, offline-first process with no dependency on a running GNS3 server. Each stage produces a well-defined intermediate artifact, enabling validation and debugging at every step. The overall flow consists of seven stages, from initial topology definition to final archive packaging.

8.1 Stage 1: Topology Skeleton Ingestion

The pipeline begins with an AI-generated topology skeleton, which is a logical description of the network topology containing node definitions (name, type, platform, role) and link definitions (source node, destination node, link type). This skeleton does not contain GNS3-specific details such as UUIDs, adapter numbers, port numbers, or coordinate positions. It represents the pure logical intent of the network designer. The ingestion stage validates the skeleton against the supported node types and link types, ensuring that no unsupported appliances or invalid connections are present. It also validates that the skeleton is internally consistent: every link references nodes that exist, no VPCS node has more than one link, and Dynamips slot configurations are compatible with the chosen platform.

8.2 Stage 2: Node Enrichment and UUID Assignment

In the second stage, each node in the skeleton is enriched with GNS3-specific properties. This involves assigning a unique UUID (`node_id`) to each node, setting the `compute_id` to "local" for portable projects, populating the `node_type` and `category` fields based on the appliance type, selecting the appropriate symbol path, and filling in all required properties for the specific node type (platform, image, ram for Dynamips; path, `application_id`, `ethernet_adapters` for IOU; etc.). Default values are applied for any optional properties not specified in the skeleton. The `console` and `console_type` fields are set to auto-allocated values. The `template_id` field may be set to null for nodes that are not derived from a template, or to a template UUID if the Structranet AI system maintains a template library.

8.3 Stage 3: Link Resolution and Port Allocation

The third stage resolves the logical links from the skeleton into concrete GNS3 link objects with fully specified endpoints. For each link, the system allocates the next available `adapter_number` and `port_number` on each endpoint node, following the port addressing rules defined in Section 5. A unique UUID (`link_id`) is assigned to each link. The `link_type` is set to "ethernet" for ethernet connections or "serial" for serial connections. Link endpoint labels are generated based on the `port_name_format` of each node, producing human-readable interface names such as "Ethernet0/0" or "FastEthernet0/1". The `filters` object is set to an empty dictionary unless the skeleton specifies packet manipulation rules such as latency, packet loss, or corruption.

8.4 Stage 4: Canvas Layout Computation

The fourth stage computes the x, y coordinates for each node and generates any drawings (text annotations, rectangles, ellipses) that should appear on the canvas. The layout algorithm should be deterministic and based on the topology structure: hierarchical topologies should be laid out vertically with core at the top, mesh topologies should use a circular layout, and linear topologies should be arranged horizontally. The z-order is set to 1 for all nodes unless explicit stacking is required. The label positions are computed relative to the node dimensions using the standard offsets defined in Section 7.3. Drawing objects are created for any text annotations or visual groupings specified in the topology skeleton.

8.5 Stage 5: Configuration Generation and Injection

The fifth stage generates the device-specific configurations and injects them into the project artifact. For Dynamips nodes, the AI-generated IOS configuration is written as `startup-config.cfg` into the `project-files/dynamips//configs/` directory. For IOU nodes, the configuration is written as `startup-config.cfg` into `project-files/iou//`. For VPCS nodes, the startup script (containing IP address assignments and other VPCS commands) is written as `startup.vpc` into `project-files/vpcs//`. For QEMU nodes, configuration is typically injected via a config disk image (`config.img`) if the `create_config_disk` option is enabled. The `startup_config_content` and `private_config_content` fields in the node properties may also be populated as a fallback mechanism, but file-system injection is the primary method because it is more reliable across different GNS3 import scenarios.

8.6 Stage 6: project.gns3 JSON Assembly

The sixth stage assembles the final `project.gns3` JSON by combining all the enriched nodes, resolved links, computed drawings, and project metadata into a single JSON object. The top-level fields are set as specified in Section 3, with `revision=9`, `type="topology"`, `computes=[]`, and `version` set to the target GNS3 version. The `topology.nodes` array contains all enriched node objects, each with its complete property set. The `topology.links` array contains all resolved link objects. The `topology.drawings` array contains any canvas annotations. The JSON is validated against the GNS3 file format schema to ensure that no required fields are missing and that all value constraints are satisfied. This validation step is critical because the GNS3 import pipeline will reject any JSON that does not conform to the schema.

8.7 Stage 7: Archive Packaging and Verification

The final stage packages the `project.gns3` JSON and all `project-files` into a ZIP archive with the `.gns3project` extension. The ZIP must follow the directory structure specified in Section 2, with `project.gns3` at the root and all node files under the correct `project-files` subdirectories. The archive must not contain excluded files (`captures`, `tmp`, `logs`, `.DS_Store`). File modification times before 1980 must be patched. After packaging, a verification pass should confirm that the archive contains a valid `project.gns3` at its root, that every `node_id` referenced in the JSON has a corresponding directory in

project-files, that all configuration files are present and non-empty, and that the ZIP can be successfully opened and its contents listed without errors. This verification step provides the final guarantee of 100% import success.

8.8 Data Flow Summary

The following table summarizes the seven-stage data flow pipeline, including the input and output artifacts at each stage and the key validation checks performed. This pipeline is designed to be executed entirely offline, with no network dependencies, and to produce a deterministic output for any given topology skeleton input.

Stage	Name	Input	Output	Key Validation
1	Topology Skeleton Ingestion	AI-generated JSON/YAML	Validated topology skeleton	Node types, link consistency, VPCS constraints
2	Node Enrichment and UUID Assignment	Validated skeleton	Enriched node objects	Mandatory fields, platform compatibility
3	Link Resolution and Port Allocation	Enriched nodes, skeleton links	Resolved link objects	Port bounds, adapter capacity, no conflicts
4	Canvas Layout Computation	Enriched nodes, resolved links	Positioned nodes, drawings	No overlapping, label readability
5	Configuration Generation and Injection	Positioned nodes, AI configs	Config files in project-files	Config syntax, file placement
6	project.gns3 JSON Assembly	All enriched artifacts	Complete project.gns3 JSON	Schema validation, required fields
7	Archive Packaging and Verification	project.gns3 + project-files	.gns3project ZIP archive	ZIP integrity, file existence, import readiness

9. Import Validation Checkpoints

Understanding the exact validation checks that GNS3 performs during import is essential for ensuring that offline-generated archives pass import without errors. The `import_project.py` module in the GNS3 server executes the following validation sequence, and failure at any checkpoint results in an immediate

import error. The Structranet AI system must ensure that its output satisfies every checkpoint.

Check point	Validation	Failure Mode	Structranet AI Mitigation
1. ZIP Integrity	Must be a valid ZIP file	BadZipFile exception	Use standard ZIP library, verify before export
2. project.gns3 Existence	project.gns3 must exist in archive root	KeyError exception	Always write project.gns3 first in archive
3. JSON Validity	project.gns3 must parse as valid JSON	ValueError exception	Validate JSON serialization, use strict encoder
4. Required Keys	project_id and name must be present	KeyError exception	Schema validation in Stage 6
5. Name Characters	Project name must not have unsupported chars	UnicodeEncodeError	Sanitize name to ASCII + common Unicode
6. Path Validation	Destination must not contain .gns3 in path	Error response	Ensure correct project directory naming
7. ID Regeneration	All node/link/drawing IDs regenerated	Silent (always succeeds)	Ensure UUIDs are valid format before packaging
8. File Migration	project-files dirs renamed to new UUIDs	FileNotFoundError	Verify all node_ids have matching dirs

10. Architectural Constraints and Recommendations

This section captures the key architectural constraints that govern the Structranet AI offline generation system and provides recommendations for ensuring robust, production-quality .gns3project generation.

10.1 Offline-First Constraint

The entire generation pipeline must operate with zero connectivity to a live GNS3 server. This means that no REST API calls can be made to verify node properties, test link validity, or compute port assignments. All validation must be performed against the schema definitions embedded in the Structranet AI system itself. The system must maintain an internal copy of the GNS3 schema constraints (platform-to-module mappings, adapter types, port limits) and validate all generated artifacts against

these constraints before packaging. This is a fundamental architectural principle that must not be violated even when a GNS3 server happens to be available; the system must produce identical output regardless of network connectivity.

10.2 Determinism Constraint

Given the same topology skeleton input, the system must always produce a byte-identical .gns3project archive. This requires deterministic UUID generation (e.g., UUID v5 based on a namespace and the node name), deterministic port allocation (sequential, never random), deterministic canvas layout (based on topology structure, not random placement), and deterministic ZIP ordering (files added in a consistent order with consistent compression settings). Determinism enables reproducible builds, simplifies testing, and allows diff-based verification of topology changes.

10.3 Schema Evolution Considerations

The GNS3 project format has evolved through 9 revisions, and future versions may introduce breaking changes. The Structranet AI system should target revision 9 (GNS3 2.2.x) as the primary format but should be architected to allow easy adaptation to future revisions. The schema definitions should be maintained as data (not hardcoded logic) so that they can be updated when new GNS3 versions are released. The system should also include a version negotiation mechanism that allows the user to specify the target GNS3 version, with appropriate schema validation applied for each version.

10.4 Error Recovery Strategy

When the system detects a validation error (e.g., an incompatible module for a platform, a port allocation overflow, or a missing mandatory field), it should not silently produce a broken archive. Instead, it should report the error with sufficient context for the user or AI system to correct the topology skeleton. Common errors and their recommended resolution strategies include: platform-module incompatibility (suggest a compatible module or a different platform), port exhaustion on a VPCS node (flag the extra link and suggest removing it or using a different node type), missing IOS image reference (warn the user that the image must be present on the target system), and canvas overlap (adjust spacing or suggest a different layout algorithm).

10.5 Testing Strategy

To ensure 100% import success, the Structranet AI system should implement a comprehensive testing strategy. Unit tests should validate each stage of the pipeline independently: node enrichment, port allocation, canvas layout, configuration injection, JSON assembly, and archive packaging. Integration tests should generate complete .gns3project archives for a variety of topology patterns (hub-and-spoke, mesh, three-tier, linear chain) and import them into a reference GNS3 installation to verify that they load without errors, that all nodes appear on the canvas, that all links are correctly connected, and that configurations are applied correctly. Regression tests should ensure that any GNS3 import failure

encountered in production results in a new test case that prevents the same failure from recurring.

Appendix A: Complete Node Object Reference

The following JSON shows a complete, GNS3-compliant node object for a Dynamips router, including all fields that appear in a topology dump. Fields not explicitly set by the Structranet AI system should be included with their default values to ensure that the JSON is complete and unambiguous. This reference can be used as a template for constructing node objects during Stage 2 of the data flow pipeline.

```
{
  "compute_id": "local",
  "console": 5000,
  "console_type": "telnet",
  "console_auto_start": false,
  "custom_adapters": [],
  "first_port_name": null,
  "height": 40,
  "label": {
    "rotation": 0,
    "style": "font-family: Typewriter;font-size: 10.0;font-weight: bold;fill: #000000;fill-opacity: 1.0;",
    "text": "R1",
    "x": -17,
    "y": -25
  },
  "locked": false,
  "name": "R1",
  "node_id": "a1e920ca-338a-4e9f-b363-aa607b09dd80",
  "node_type": "dynamips",
  "port_name_format": "Ethernet{0}",
  "port_segment_size": 0,
  "properties": {
    "platform": "c3725",
    "image": "c3725-adventerprisek9-mz.124-15.T14.image",
    "ram": 256,
    "nvram": 256,
    "slot0": "GT96100-FE",
    "slot1": "NM-1FE-TX",
    "wic0": null,
    "wic1": null,
    "wic2": null,
    "startup_config": "ios_base_startup-config.txt",
    "private_config": "",
    "idlepc": "",
    "mac_addr": "",
    "mmap": true,
    "sparsemem": true,
    "auto_delete_disks": false
  },
  "symbol": ":/symbols/router.svg",
  "template_id": null,
}
```

```
"width": 56,  
"x": -200,  
"y": -100,  
"z": 1  
}
```

Appendix B: Complete Link Object Reference

The following JSON shows a complete, GNS3-compliant link object. Each link connects exactly two nodes via their endpoints, where each endpoint specifies the `node_id`, `adapter_number`, and `port_number`. The `link_id` and `project_id` are UUIDs that must be unique and valid, respectively. The `filters` object can contain packet manipulation rules for simulating network impairments such as latency, packet loss, and corruption.

```
{  
  "filters": {},  
  "link_id": "b2f0abcd-1234-5678-abcd-123456789012",  
  "link_style": {  
    "color": "#000000",  
    "width": 2,  
    "type": 0  
  },  
  "link_type": "ethernet",  
  "nodes": [  
    {  
      "adapter_number": 0,  
      "label": {  
        "rotation": 0,  
        "style": null,  
        "text": "FastEthernet0/0",  
        "x": -10,  
        "y": -10  
      },  
      "node_id": "a1e920ca-338a-4e9f-b363-aa607b09dd80",  
      "port_number": 0  
    },  
    {  
      "adapter_number": 0,  
      "label": {  
        "rotation": 0,  
        "style": null,  
        "text": "Ethernet0/0",  
        "x": 10,  
        "y": -10  
      },  
      "node_id": "ffffffff-0000-1111-2222-333333333333",  
      "port_number": 0  
    }  
  ],  
}
```

```
"project_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"suspend": false
}
```

Appendix C: Dynamips Valid Adapter Modules

The following table lists all valid adapter modules for Dynamips nodes, organized by the platform families they support. The Structranet AI system must only assign modules that are compatible with the selected platform; otherwise, GNS3 will reject the node during creation. Port Adapters (PA-*) are specific to the c7200 platform, while Network Modules (NM-*) are used by c3725, c3745, c2691, c3600, c2600, and c1700 platforms. WIC modules provide additional interfaces on platforms that support WIC slots.

Module Type	Module Name	Ports	Interface Type	Compatible Platforms
I/O Controller	C7200-IO-FE	1	FastEthernet	c7200 (slot 0)
I/O Controller	C7200-IO-2FE	2	FastEthernet	c7200 (slot 0)
I/O Controller	C7200-IO-GE-E	1	GigabitEthernet	c7200 (slot 0)
Port Adapter	PA-FE-TX	1	FastEthernet	c7200 (slots 1-6)
Port Adapter	PA-2FE-TX	2	FastEthernet	c7200 (slots 1-6)
Port Adapter	PA-GE	1	GigabitEthernet	c7200 (slots 1-6)
Port Adapter	PA-4T+	4	Serial	c7200 (slots 1-6)
Port Adapter	PA-8T	8	Serial	c7200 (slots 1-6)
Network Module	NM-1FE-TX	1	FastEthernet	c3725, c3745, c2691, c3600
Network Module	NM-16ESW	16	FastEthernet (switch)	c3725, c3745, c2691, c3600
Network Module	NM-4E	4	Ethernet	c3725, c3745, c2691, c3600
Network Module	NM-4T	4	Serial	c3725, c3745, c2691, c3600

Network Module	NM-1E	1	Ethernet	c3725, c3745, c2691, c3600
Built-in	GT96100-FE	2	FastEthernet	c3725 (motherboard, adapter 0)
Built-in	Leopard-2FE	2	FastEthernet	c3745 (motherboard, adapter 0)
WIC	WIC-1ENET	1	Ethernet	c3725, c3745, c2691, c2600
WIC	WIC-1T	1	Serial	c3725, c3745, c2691, c2600
WIC	WIC-2T	2	Serial	c3725, c3745, c2691, c2600